

Modelos Generativos Profundos

Clase 6: Modelos autorregresivos y LLMs (parte II)

Fernando Fêtis Riquelme

Primavera, 2025

Facultad de Ciencias Físicas y Matemáticas

Universidad de Chile

Arquitectura GPT

Arquitectura GPT

Layer normalization

- La aplicación consecutiva de varias capas neuronales suele afectar el entrenamiento de la red (p.g. por **vanishing gradient** o **exploding gradient**).
- Esto se puede mitigar agregando **capas de normalización** entre medio de los bloques de la red.
- Por ejemplo, en redes convolucionales es usual utilizar **batch normalization** o **group normalization**.
- No es obvio cómo aplicar batch norm en modelos secuenciales: las secuencias de tokens no suelen tener el mismo largo y muchas veces los batches de entrenamiento deben ser pequeños.

Layer normalization

Para una secuencia de embeddings, **layer normalization** normaliza cada vector de embedding de forma individual.

Para un vector de embedding de la secuencia, $x \in \mathbb{R}^D$, primero se obtiene el embedding normalizado $\tilde{x} \in \mathbb{R}^D$, donde:

$$\tilde{x}_d = \frac{x_d - \mu}{\sigma}, \quad \text{para} \quad \mu = \frac{1}{D} \sum_{d=1}^D x_d \quad \text{y} \quad \sigma^2 = \frac{1}{D} \sum_{d=1}^D (x_d - \mu)^2$$

Luego, de forma análoga a lo realizado por batch norm, layer norm aplica una transformación coordenada a coordenada:

$$\text{LayerNorm}(x) = \alpha \odot \tilde{x} + \beta,$$

donde $\alpha, \beta \in \mathbb{R}^D$ son parámetros entrenables.

Self-attention

Dada una secuencia de embeddings, $(x_1, \dots, x_T) \subset \mathbb{R}^D$:

- El objetivo del **mecanismo de auto-atención** es obtener una secuencia de *embeddings mejorados*, $(y_1, \dots, y_T) \subset \mathbb{R}^P$.
- Cada **embedding contextual** $y_t \in \mathbb{R}^P$ combina el respectivo embedding $x_t \in \mathbb{R}^D$ con todos los otros vectores de embedding, $(x_1, \dots, x_T)$, para poder incorporar información contextual.
- Esto soluciona el problema de **memoria a largo plazo** presente en las redes recurrentes.
- Además, el procesamiento de una secuencia de embeddings es **paralelizable**, lo que permite escalar el entrenamiento. Esto no es posible con las redes recurrentes.

Self-attention

- Se comienza proyectando cada vector de embeddings de la secuencia $(x_1, \dots, x_T) \subset \mathbb{R}^D$ a $\mathbb{R}^P$. Es necesario proyectar 3 veces:

$$q_t = W_Q x_t, \quad k_t = W_K x_t, \quad v_t = W_V x_t, \quad \forall t \in \{1, \dots, T\}.$$

- $W_Q, W_K, W_V \in \mathcal{M}_{P,D}(\mathbb{R})$ son parámetros entrenables.
- Cada embedding mejorado $y_i \in \mathbb{R}^P$ se calcula como

$$y_i = \sum_{j=1}^T a_{ij} v_j, \quad \text{donde} \quad a_{ij} \geq 0 \text{ y } \sum_{j=1}^T a_{ij} = 1$$

- $a_{ij} \in [0, 1]$ representa la **atención** que coloca el token i al token j . Una elección típica es **scaled dot-product attention**:

$$a_{ij} = \frac{e^{s_{ij}}}{\sum_{t=1}^T e^{s_{it}}} \quad \text{donde} \quad s_{ij} = \frac{\langle q_i, k_j \rangle}{\sqrt{P}}$$

Self-attention

Para paralelizar el cálculo de $(y_1, \dots, y_T) \in \mathbb{R}^P$ es necesario guardar las proyecciones en matrices $Q, K, V \in \mathcal{M}_{T,P}(\mathbb{R})$:

$$Q = \begin{pmatrix} q_1^\top \\ \vdots \\ q_T^\top \end{pmatrix}, \quad K = \begin{pmatrix} k_1^\top \\ \vdots \\ k_T^\top \end{pmatrix}, \quad V = \begin{pmatrix} v_1^\top \\ \vdots \\ v_T^\top \end{pmatrix},$$

Dado que $(QK^\top)_{ij} = \langle q_i, k_j \rangle$, entonces $s_{ij} = \left(\frac{QK^\top}{\sqrt{P}} \right)_{ij}$. Además, notando que $A = \text{softmax}_{\text{por filas}}(S)$, se tiene que:

$$\begin{pmatrix} y_1^\top \\ \vdots \\ y_T^\top \end{pmatrix} = \text{softmax}_{\text{por filas}} \left(\frac{QK^\top}{\sqrt{P}} \right) V \in \mathcal{M}_{T,P}(\mathbb{R})$$

Masked self-attention

- La auto-atención anterior pierde la **causalidad** del lenguaje al momento de computar los embeddings contextuales $(y_1, \dots, y_T)$.
- Cada embedding contextual $y_t \in \mathbb{R}^P$ solo debe poder acceder a los embeddings $x_1, \dots, x_t$ y no a los embeddings futuros, $x_{t+1}, \dots, x_T$.
- Por lo tanto, se debe **enmascarar** el futuro forzando que $a_{ij} = 0$ para $j > i$.
- No se puede simplemente sustituir $a_{ij} = 0$ para $j > i$ ya que se dejaría de cumplir $\sum_{j=1}^T a_{ij} = 1$.
- El cambio se debe realizar previo al softmax: en $S \in \mathcal{M}_{T,T}(\mathbb{R})$ sustituir cada entrada donde $j > i$ por un valor muy negativo.

Multihead self-attention

- En las redes convolucionales es usual utilizar varios filtros para capturar diferentes patrones.
- De forma análoga, en el mecanismo de auto-atención se pueden utilizar múltiples cabezales de atención para capturar diferentes relaciones entre los tokens.

Si $(x_1, \dots, x_T) \subset \mathbb{R}^D$ es una secuencia de vectores de embeddings e $(y_1^{(h)}, \dots, y_T^{(h)}) \subset \mathbb{R}^P$ es la salida de la h -ésima cabeza de atención, entonces pueden concatenar

$$\tilde{y}_t = \left(y_t^{(1)} \cdots y_t^{(H)} \right) \in \mathbb{R}^{P \cdot H}, \quad \text{para } t \in \{1, \dots, T\}$$

Los embeddings finales que retorna un MHSA son de la forma $u_t = W_0 \tilde{y}_t \in \mathbb{R}^D$, donde la matriz $W_0 \in \mathcal{M}_{P \cdot H, D}(\mathbb{R})$ proyecta los embeddings de vuelta al espacio $\mathbb{R}^D$.

Red feedforward

El último paso que realiza un bloque Transformer es pasar cada uno de los embeddings contextuales, $(u_1, \dots, u_T) \subset \mathbb{R}^D$, por una misma MLP de dos capas. Para $t \in \{1, \dots, T\}$:

$$\text{FF}(u_t) = W_2 \phi(W_1 u_t),$$

- $W_1 \in \mathcal{M}_{J,D}(\mathbb{R})$, $W_2 \in \mathcal{M}_{D,J}(\mathbb{R})$ son los parámetros de la red.
- $\phi : \mathbb{R} \rightarrow \mathbb{R}$ es una función de activación que se aplica coordenada a coordenada.
- Es usual considerar $J \gg D$ para darle más capacidad de representación a la red.

Bloque Transformer

Si secuencia de embeddings, $(x_1, \dots, x_T) \subset \mathbb{R}^D$, se almacena en una matriz $X \in \mathcal{M}_{T,D}(\mathbb{R})$, un **bloque Transformer** tiene la siguiente estructura:

$$\begin{cases} Y = X + \text{Dropout}(\text{MHSA}(\text{LayerNorm}(X))) \\ \text{TransformerBlock}(X) = Y + \text{Dropout}(\text{FF}(\text{LayerNorm}(Y))) \end{cases}$$

Las capas de normalización y la red feed forward se aplican de forma individual sobre cada vector de embedding (i.e., por filas).

Con esto, la arquitectura GPT realiza lo siguiente:

- Recibe una secuencia de tokens de entrada, $(w_1, \dots, w_T) \in \mathcal{V}^*$.
- Esta secuencia es pasada por una capa de embedding inicial.
- El Transformer es **equivariante bajo permutación**, por lo que se debe sumar un **positional encoding** a cada embedding.
- Estos embeddings iniciales son pasados por varios bloques Transformer consecutivos.
- Finalmente, se agrega un cabezal de lenguaje que cambia la dimensión de la salida a la dimensión del vocabulario.

En la próxima clase.

- Revisión de variaciones de la arquitectura Transformer.

Modelos Generativos Profundos

Clase 6: Modelos autorregresivos y LLMs (parte II)